The **Collection Framework** in Java is a set of classes and interfaces that implement commonly reusable data structures. It was introduced in Java 1.2 and provides a unified architecture to store and manipulate groups of objects.

In Java, a **Collection** is a group of objects, known as elements and is represented as a single entity. With the Collection Framework, developers can manage data collections efficiently, supporting operations like searching, sorting, insertion, manipulation, and deletion. Collections are also called containers in Java.

The concept is similar to the Standard Template Library (STL) and containers of C++;

With the introduction of generics since JDK5, there is type safety and an element of a different data type cannot be stored in a collection because the relevant method will throw an exception. Only one type of object may be stored in a collection.

Collection framework consists of:

- **Interfaces** A hierarchy of *interfaces* that contains abstract methods.
- **Classes** These provide concrete implementation of interfaces.
- **Algorithms** These are the generic methods for performing computational work and other manipulations on collections.
- **Maps** Maps are the collections that store data in terms of key-value pairs. The first value is the key and the second is the value associated with the key.

SURESH YADLAPATI

Purpose of Collection Framework:

- **Standardized Data Structures:** The Collection Framework provides pre-built, standardized data structures (like lists, sets, queues, and maps), saving developers from writing custom data structures.
- **Code Reusability and Maintainability**
- **Unified API for Data Operations**: The Collection Framework standardizes methods like add, remove, contains, size, and iterator across different types of collections.
- **Increased Performance and Efficiency:** allows quick access, fast searching, or efficient sorting

Application of Collection Framework:

The framework provides following facilities:

- Construction of different types of collections for a variety of purposes.
- Add an element to a collection or remove an element from a collection.

- It provides access to each element of a collection.
- It provides a facility to search for an element in a collection.
- Sort the collection according to some criterion

Hierarchy of collection Interfaces / classes:

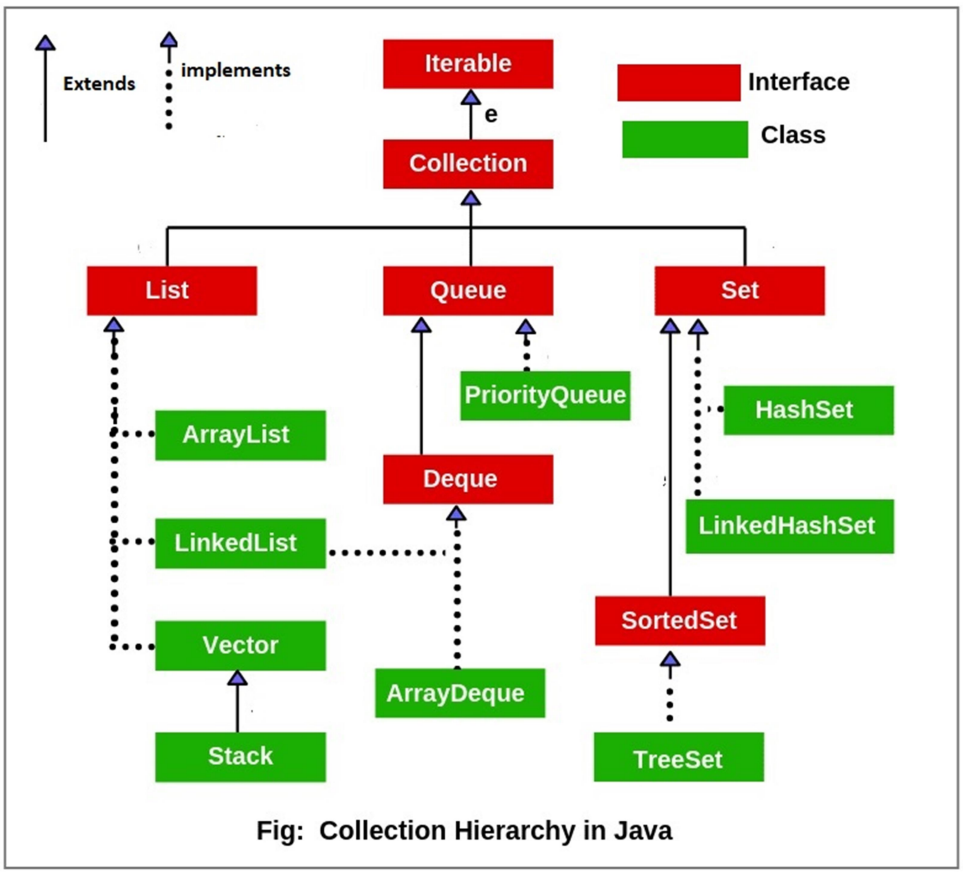


Fig: Collection Hierarchy in Java

I. **Iterable Interface**

- The root interface for all collection classes and interfaces.
- Provides an iterator() method to traverse elements in a collection.

II. **Collection Interface**

- Extends Iterable and is the base interface for most collections.
- Defines basic methods like add(), remove(), size(), and clear().

III**. Sub-Interfaces of Collection**

1. **List Interface**
    - An ordered collection (insertion order is preserved) that allows duplicate elements.

- Elements can be accessed by their index.
- Implementations:
  - ArrayList: Resizable array; allows fast random access.
  - LinkedList: Doubly linked list; efficient for insertions and deletions.
  - Vector: Synchronized, resizable array;
  - Stack: LIFO data structure; extends Vector.

2. **Set Interface**
   - A collection that does not allow duplicate elements. Also insertion order is not preserved.
   - Implementations:
     - HashSet: Backed by a hash table; does not maintain order.
     - LinkedHashSet: Maintains insertion order; backed by a hash table and linked list.
3. **SortedSet interface**:

   The SortedSet interface in Java is a sub interface of Set that provides additional methods to handle elements in sorted (ascending) order. Elements in a SortedSet must implement the Comparable interface or be ordered by a provided Comparator.
   - Implementations:
     - TreeSet: Maintains elements in ascending order.
4. **Queue Interface:**
   - Typically represents a collection that holds elements prior to processing.
   - Most queues follow First-In-First-Out (FIFO) order.
   - Implementations:
     - PriorityQueue: Elements are ordered based on natural ordering or user defined priority.

5. **Deque Interface** (Subinterface of Queue)
   - Extends Queue and allows elements to be added or removed from both ends.
   - Implementations:
     - ArrayDeque: Array-based double-ended queue.
     - LinkedList: Can act as a FIFO queue when used as a Queue.

SURESH YADLAPATI

Map Interface:

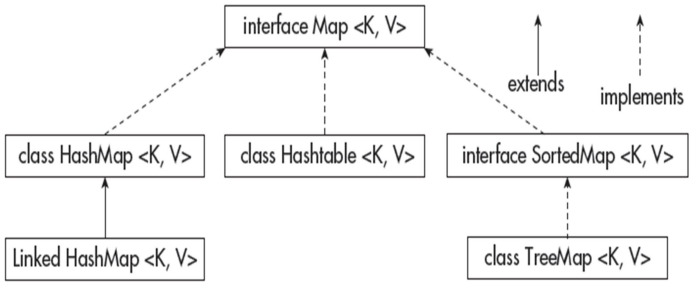


Fig. 16.2 Hierarchy of maps

- The Map interface in Java is part of the Java Collections Framework and provides a way to store data in key-value pairs.
- Unlike other collection interfaces (Collection, Set, List), Map does not inherit from Collection because it is designed specifically for mapping unique keys to values rather than holding individual elements.
- Implementations:
    - HashMap: Hash table-based implementation; allows one null key.
    - LinkedHashMap: Maintains insertion order.
    - Hashtable: Synchronized hash table; does not allow null keys or values.
    - Interface SortedMap: Maintains its entries in ascending order of the keys
        - TreeMap: Red-Black tree-based implementation; maintains ascending order.

SURESH YADLAPATI

Summary of Core Interfaces and Their Classes:

Interface	Description	Implementation Classes
List	Ordered collection, allows duplicates	ArrayList, LinkedList, Vector, Stack
Set	Unordered collection, no duplicates	HashSet, LinkedHashSet
SortedSet	Ordered collection, no duplicates	TreeSet
Queue	FIFO ordering, holds elements for processing	LinkedList, PriorityQueue,
Deque	Double-ended queue	ArrayDeque, LinkedList
Map	Key-value pairs, unique keys	HashMap, LinkedHashMap, TreeMap, Hashtable

Iterator interface:

- Java **Iterator** Interface of java collections allows us to access elements of the collection and is used to iterate over the elements in the collection (Ex: List or Set).
- **Iterator** is a universal iterator as it can be applied to any Collection object.
- We can traverse only in the **forward** direction using iterator.
- Almost all collections support iterators except the maps. In maps, the elements are accessed through keys.
- Iterator interface is used in Java Collections framework in the place of Enumeration.
    - Enumeration works only on legacy classes such as Vector and Hashtable.
    - Removing element the given collection during iterating over the elements is not possible in enumeration.
    - Only forward direction is possible in an Enumeration.

Methods:

Method	Purpose
boolean hasNext()	Checks if more elements are present or not
E next()	Retrieves the next element
void remove()	Removes the last element returned by the next() method from the collection.

SURESH YADLAPATI

ListIterator: Extends Iterator to allow bidirectional traversal of a collection (both forward and backward). Primarily used in list implementations like ArrayList and LinkedList, where both forward and backward traversal is needed, or where elements might need to be replaced or added during iteration.

Additional methods in ListIteraor interface:

- **boolean hasPrevious()**: Checks if there is an element before the current element in the list.
- **E previous()**: Returns the previous element in the list.
- **int nextIndex()**: Returns the index of the element that would be returned by next().
- **int previousIndex()**: Returns the index of the element that would be returned by previous().
- **void set(E e)**: Replaces the last element returned by next() .
- **void add(E e)**: Inserts the specified element into the list at the iterator's current position.

Methods declared in Collection interface:

- The Collection interface represents a unified architecture for performing different operations on a group of objects such as searching, sorting, insertion, manipulation, and deletion.
- The methods defined in this interface are inherited by all sub-interfaces and implemented by sub classes.

Method	Description
boolean add(Object obj)	Adds obj to the invoking collection. Returns true if obj was added to the collection. Returns false if obj is already a member of the collection, or if the collection does not allow duplicates.
boolean addAll(Collection c)	Adds all the elements of c to the invoking collection. Returns true if the operation succeeds (i.e., the elements were added). Otherwise, returns false.
void clear( )	Removes all elements from the invoking collection.
boolean contains(Object obj)	Returns true if obj is an element of the invoking collection. Otherwise, returns false.
boolean containsAll(Collection c)	Returns true if the invoking collection contains all elements of **c**. Otherwise, returns false.
boolean equals(Object obj)	Returns true if the invoking collection and obj are equal. Otherwise, returns false.
int hashCode( )	Returns the hash code for the invoking collection.
boolean isEmpty( )	Returns true if the invoking collection is empty. Otherwise, returns false.
Iterator iterator( )	Returns an iterator for the invoking collection.
boolean remove(Object obj)	Removes one instance of obj from the invoking collection. Returns true if the element was removed. Otherwise, returns false.
boolean removeAll(Collection c)	Removes all elements of c from the invoking collection. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false.
boolean retainAll(Collection c)	Removes all elements from the invoking collection except those in c. Returns true if the collection changed (i.e., elements were removed). Otherwise, returns false.
int size( )	Returns the number of elements held in the invoking collection

SURESH YADLAPATI

Object[ ] toArray( )	Returns an array that contains all the elements stored in the invoking collection. The array elements are copies of the collection elements.
<T>[ ] toArray (T array[ ])	Returns an array containing only those collection elements whose type matches that of array.

ArrayList Class:

The ArrayList class in Java is part of the java.util package and is a widely used data structure. It implements the List interface, allowing you to store dynamically resizable arrays, unlike regular arrays which have a fixed size.

Key features:

- Automatically resizes itself when elements are added or removed.
- We can specify the type of elements stored using generics (ArrayList<String> or ArrayList<Integer>).
- Elements are accessed and manipulated based on their index, starting at 0.
- ArrayList is not thread-safe;
- ArrayList is the best choice for efficient retrieval purpose (random access is possible), not for insertion or deletion because several shifting operations are required

SURESH YADLAPATI

Syntax:

public class ArrayList<E> extends AbstractList<E> implements List<E>, RandomAccess, Cloneable, Serializable

Serialization in programming refers to the process of converting an object or data structure into a format that can be easily stored (e.g., in a file or database) or transmitted (e.g., over a network) and later reconstructed (deserialized) into its original form.

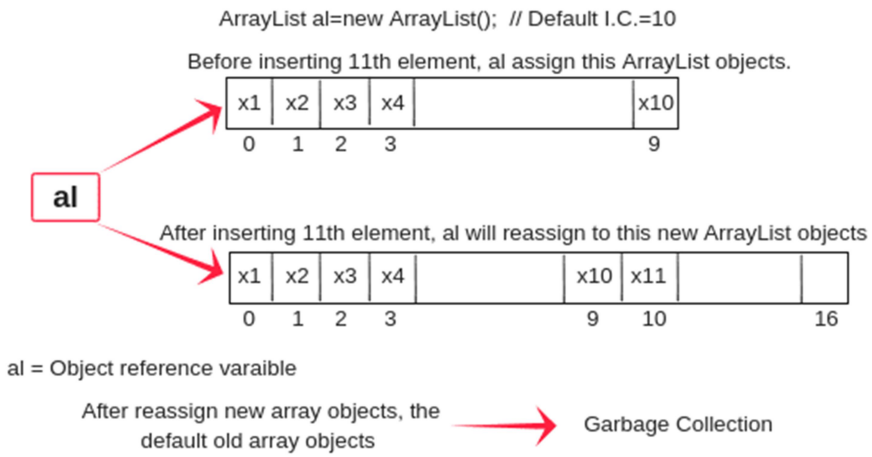
Constructors:

1. **ArrayList():** This constructs an empty list with initial default capacity of 10.
2. **ArrayList:** (int Capacity) This constructor method constructs an empty list having a specified initial capacity.
3. **ArrayList:** (Collection *Collection*)

    - It constructs a list with elements of a specified collection.
    - The elements are kept in the order in which they are delivered to ArrayList instance.

Ex:

ArrayList al = new ArrayList();



- Once ArrayList is reached its maximum capacity, the ArrayList class automatically creates a new array with a larger capacity.
- **New capacity** = (current capacity*3/2) + 1 = 10*3/2 + 1 = 16
- After creating a new array list with a larger capacity, all the existing elements are copied into the new array list and then adds the new element into it.

SURESH YADLAPATI

Methods:

1. Adding Elements

- add(E e): Adds the specified element to the end of the list.
- add(int index, E element): Inserts the specified element at the specified position in the list.
- addAll(Collection c) – adds the collection at the end

2. Accessing Elements

- get(int index): Returns the element at the specified position in the list.

3. Updating Elements

- set(int index, E element): Replaces the element at the specified position with the specified element.

4. Removing Elements

- remove(int index): Removes the element at the specified position in the list.
- remove(Object o): Removes the first occurrence of the specified element from the list.
- removeAll(Collection) – removes all the lements specified in the collection

- retailnAll(Collection c) – only the specified elements in the collection will be retained

5. Searching and Checking Existence

- contains(Object o): Returns true if the list contains the specified element.
- indexOf(Object o): Returns the index of the first occurrence of the specified element, or -1 if not found.
- lastIndexOf(Object o): Returns the index of the last occurrence of the specified element, or -1 if not found.

6. Size and Capacity

- size(): Returns the number of elements in the list.
- isEmpty(): Returns true if the list contains no elements.
- ensureCapacity(int minCapacity): Increases the capacity of the ArrayList instance.

7. Iterating and Traversing

- iterator(): Returns an iterator over the elements in the list.
- listIterator(): Returns a list iterator for traversing the list in either direction.

8. Other Utility Methods

- clear(): Removes all elements from the list.
- toArray(): Converts the list to an array.
- subList(int fromIndex, int toIndex): Returns a view of a portion of the list.

SURESH YADLAPATI

Example Program on ArrayList: to store an array of strings:

```
import java.util.*;
public class ArrayListEx {
   public static void main(String[] args) {
      ArrayList<String> fruits = new ArrayList<>();// Creating an ArrayList of Strings
      // Adding elements to the ArrayList
      fruits.add("Apple");
      fruits.add("Banana");
      fruits.add("Cherry");
      fruits.add(1, "Blueberry"); // Adding an element at a specific index

      // Printing the ArrayList
      System.out.println("Fruits list: " + fruits); // [Apple, Blueberry, Banana, Cherry]

     // Accessing an element by index
      String fruit = fruits.get(2);
      System.out.println("Fruit at index 2: " + fruit); // Banana
```

```
    // Updating an element at a specific index
        fruits.set(2, "Mango");
        System.out.println("After update: " + fruits); // [Apple, Blueberry, Mango,
Cherry]

       // Removing an element by index
        fruits.remove(1);
        System.out.println("After removing element at index 1: " + fruits); // [Apple,
Mango, Cherry]

       // Checking if an element is in the list
        System.out.println("Contains Cherry? " = fruits.contains("Cherry")); // true

        // Getting the size of the ArrayList
        int size = fruits.size();
        System.out.println("Size of the list: " + size); // 3

       // Iterating over elements using a for-each loop
        System.out.print("Fruits: ");
        for (String f : fruits) {
            System.out.print(f + " ");
        }

       // Printing the ArrayList using an Iterator
        System.out.print("Fruits list using Iterator: ");
        Iterator<String> i = fruits.iterator();
        while (i.hasNext()) {
            System.out.print(i.next() + " ");
        }
       // Clearing all elements in the ArrayList
        fruits.clear();
        System.out.println("\nAfter clearing the list: " + fruits); // prints  []

    }
}
```

SURESH YADLAPATI

Develop a Java program to merge two arraylists into one list:

```
import java.util.ArrayList;

public class ArrayListMergeEx  {
   public static void main(String[] args) {
      // Creating the first ArrayList
      ArrayList<String> list1 = new ArrayList<>();
      list1.add("Apple");
      list1.add("Banana");
      list1.add("Cherry");

      // Creating the second ArrayList
      ArrayList<Integer> list2 = new ArrayList<Integer>();
      list2.add(1);
      list2.add(2);
      list2.add(3);

      // Creating a new ArrayList to hold the merged lists
      ArrayList<Object> mergedList = new ArrayList<Object>();
      mergedList.addAll(list1);
      mergedList.addAll(list2);
      System.out.println("Merged List: " + mergedList);
    }
}
```

Output:

Merged List: [Apple, Banana, Cherry, 1, 2, 3]

SURESH YADLAPATI

Develop a Java program that illustrates the usage of the ArrayList class to store, manipulate, and display objects of a custom class Employee.

```
import java.util.*;
class Employee {
        int id;
        String name;
        double salary;
        // Constructor
        Employee(int id, String name, double salary) {
           this.id = id;
           this.name = name;
           this.salary = salary;
        }
```

```
        @Override
         public String toString() {
            return "Employee [ID=" + id + ", Name=" + name + ", Salary=" + salary +
      "]";
         }
}
public class ArrayList_Class {
   public static void main(String[] args) {
                // Creating an ArrayList of Employee objects
                ArrayList<Employee> e = new ArrayList<>();
                // Adding Employee objects to the ArrayList
                e.add(new Employee(1, "Saadvi", 50000));
                e.add(new Employee(2, "Suresh", 60000));
                e.add(new Employee(3, "Kalpana", 55000));

                // Displaying all employees using a for-each loop
                System.out.println("Employee List:");
                for (Employee emp : e) {
                   System.out.println(emp);
                }
                // Adding another Employee
                e.add(new Employee(4, "Poojitha", 70000));
                // Accessing a specific employee(s_e) by index
                Employee s_e = e.get(1);
                System.out.println("\nEmployee at index 1: " + s_e);

                // Removing an employee by index
                e.remove(2);
                //After Removing
                Iterator<Employee> i = e.iterator();
                while (i.hasNext()) {
                                System.out.println(i.next());
                        }
        }
}
```

Output:

```
Employee List:
Employee [ID=1, Name=Saadvi, Salary=50000.0]
Employee [ID=2, Name=Suresh, Salary=60000.0]
Employee [ID=3, Name=Kalpana, Salary=55000.0]

Employee at index 1: Employee [ID=2, Name=Suresh, Salary=60000.0]
```

After Removing

```
Employee [ID=1, Name=Saadvi, Salary=50000.0]
Employee [ID=2, Name=Suresh, Salary=60000.0]
Employee [ID=4, Name=Poojitha, Salary=70000.0]
```

SURESH YADLAPATI

HashSet Class:

Java HashSet class is used to create a collection that uses a HashMap i.e., hash table for storage. It inherits the AbstractSet class and implements Set interface.

Syntax:
public class HashSet<e> extends AbstractSet<e> implements Set<e>, Cloneable, Serializable

Key points of HashSet class are:

- HashSet stores the elements by using a mechanism called hashing.
- HashSet contains unique elements only(no duplicates).
- HashSet allows null value.
- HashSet class is non-synchronized (not thread safe).
- HashSet doesn't maintain the insertion order. Here, elements are inserted on the basis of their hashcode.

Constructors:

- HashSet h = new HashSet(); is the default constructor. Default initial capacity is 16, load factor is 0.75.
- HashSet h = new HashSet(int initialCapacity); is a constructor with a given initial capacity. The load factor is 0.75.
- HashSet h = new HashSet(int initialCapacity, float loadFactor); is a constructor with a given initial capacity and load factor.
- HashSet h = new HashSet(Collection C); is a constructor that adds elements from another collection.
- The new capacity after resizing is usually: **New Capacity=Old Capacity×2;**

Methods:

- **boolean add(E e):** adds an element to HashSetif it doesn't exist, if it already exists, the method returns false .
- **void clear():**removes all elements from the set.
- **boolean contains(Object o):** Returns true if the given element is present in the set.
- **boolean remove(Object o):** Removes the given element from the set, if any.
- **Iterator iterator():** returns an iterator for the elements of the set.
- **boolean isEmpty():** returns true if the set has no elements.
- **Object clone():** performs a shallow clone HashSet.

Example Program:

```
import java.util.*;
class HashSetExample1{
 public static void main(String args[]){
        //Creating HashSet and adding elements
         HashSet<string> set=new HashSet();
        set.add("One");
        set.add("Two");
        set.add("Three");
        set.add("Four");
        set.add("Five");

        Iterator<string> i=set.iterator();
        while(i.hasNext())
        {
                System.out.println(i.next());
        }
       }
}
```

Example Program to merging of two hash sets into one hash set:

```
import java.util.HashSet;

public class HashSetMergeEx {
   public static void main(String[] args) {
      // Creating the first HashSet
      HashSet<String> hs1 = new HashSet<>();
      hs1.add("Apple");
      hs1.add("Banana");
      hs1.add("Cherry");

      // Creating the second HashSet
      HashSet<Integer> hs2 = new HashSet<Integer>();
      hs2.add(1);
      hs2.add(2);
      hs2.add(3);

      // Creating a new HashSet to hold the merged lists
      HashSet<Object> mergedset = new HashSet<Object>();
      mergedset.addAll(hs1);
      mergedset.addAll(hs2);

      System.out.println("Merged Set: " + mergedset);
   }
}
```

Output: Merged Set: [1, Apple, 2, 3, Cherry, Banana]

SURESH YADLAPATI

Develop a Java program that illustrates the usage of the HashSet class to store, and display objects of a custom class Employee.

```
import java.util.*;
class Employee {
   int id;
   String name;
   double salary;
   // Constructor
   Employee(int id, String name, double salary) {
      this.id = id;
      this.name = name;
      this.salary = salary;
   }
   @Override
   public String toString() {
      return "Employee [ID=" + id + ", Name=" + name + ", Salary=" + salary + "]";
   }
}
public class HashSet_Class {
   public static void main(String[] args) {
      // Creating an HashSet of Employee objects
      HashSet<Employee> e = new HashSet<>();
      // Adding Employee objects to the HashSet
      e.add(new Employee(1, "Saadvi", 50000));
      e.add(new Employee(2, "Suresh", 60000));
      e.add(new Employee(3, "Kalpana", 55000));

      Iterator<Employee> i = e.iterator();
      while (i.hasNext()) {
               System.out.println(i.next());
       }
   }
}
```

SURESH YADLAPATI

Output:

```
Employee [ID=3, Name=Kalpana, Salary=55000.0]
Employee [ID=1, Name=Saadvi, Salary=50000.0]
Employee [ID=2, Name=Suresh, Salary=60000.0]
```

Priority Queue:

- The PriorityQueue class in Java is part of the java.util package.
- It is a collection that processes elements based on their priority.
- By default, it uses a **min-heap**, meaning the smallest element has the highest priority.

  It can also be customized to use a comparator for custom ordering.
- Key points:

  1. The elements are ordered according to their natural order or according to the comparator provided at the time of creation.

  2. The null elements are not permitted.

  3. The priority queue working on natural order does not permit the insertion of non-comparable objects.

Syntax:

The class is declared as
public class PriorityQueue <E> extends AbstractQueue <E> implements Queue, serializable

SURESH YADLAPATI

Constructors:

- **PriorityQueue<E> pq = new PriorityQueue<>();:** Creates an empty priority queue with the default initial capacity (11) and natural ordering.
- **PriorityQueue<E> pq = new PriorityQueue<>(int capacity); :** Creates a priority queue with the specified initial capacity and natural ordering.
- **PriorityQueue<E> pq = new PriorityQueue<>(Comparator<E> comparator); :** Creates a priority queue with a custom comparator for ordering elements.
- **PriorityQueue<E> pq = new PriorityQueue<>(int initialCapacity, Comparator<E> comparator); :** Creates a priority queue with both a specified capacity and custom comparator.
- **PriorityQueue<E> pq = new PriorityQueue<>(Collection<E> c); :** Creates a priority queue containing the elements of the specified collection, using natural ordering.

Methods:

- **boolean add(E e);** - Inserts the specified element into the queue. Throws an exception if the queue is full.
- **boolean offer(E e);** - Inserts the specified element into the queue. Returns false if it cannot be added (unlike add, it doesn't throw an exception).
- **E poll();** - Retrieves and removes the head of the queue. Returns null if the queue is empty.
- **E peek();** - Retrieves the head of the queue without removing it. Returns null if the queue is empty.

- **boolean remove(Object o);** - Removes a single instance of the specified element from the queue, if it exists.
- **boolean contains(Object o);** - Returns true if the queue contains the specified element.
- **void clear();** - Removes all elements from the queue.
- **int size();** - Returns the number of elements in the queue.
- **boolean isEmpty();** - Returns true if the queue contains no elements.
- **Iterator iterator();** – returns iterator instance.
- **Object[] toArray();** - Returns an array containing all elements in the queue.

Ex:

```
import java.util.*;
public class PriorityQueueEx {
   public static void main(String[] args) {
      // Create a PriorityQueue with natural ordering
      Queue<Integer> pq = new PriorityQueue<>();
      // Add elements to the PriorityQueue
      pq.add(10);
      pq.add(5);
      pq.add(20);
                              System.out.println("Elements:"+pq);
      // Retrieve the head of the queue
      System.out.println("Head Element: " + pq.peek()); // Output: 5
       // Remove the head of the queue
      System.out.println("Removed Element: " + pq.poll()); // Output: 5
     // Check size
      System.out.println("Size: " + pq.size()); // Output: 2
      System.out.println("Elements:"+pq);

   }
}
```

SURESH YADLAPATI

Ex 2: Merging of two priority queues

```
import java.util.*;
public class PriorityQueue_Het {
   public static void main(String[] args) {
         // Create a PriorityQueue with natural ordering
         PriorityQueue<Integer> pq1 = new PriorityQueue<>();
         // Add elements to the PriorityQueue
         pq1.add(10);
         pq1.add(5);
         pq1.add(20);
                              System.out.println("First PQ Elements:"+pq1);
         PriorityQueue<String> pq2 = new PriorityQueue<>();
         // Add elements to the PriorityQueue
```

```
            pq2.add("Banana");
            pq2.add("Apple");
            pq2.add("Cherry");
        System.out.println("Second PQ Elements:"+pq2);
        PriorityQueue<Object> pq3 = new PriorityQueue<>();
        pq3.addAll(pq1);
        pq3.addAll(pq2);
        System.out.println("Merged PQ Elements:"+pq3);

    }
}
```

Ex 3:

```
import java.util.*;
public class PriorityQueue_Class {
   public static void main(String[] args) {
// Custom comparator to prioritize employees by salary (higher salary = higher
priority)
Comparator<Employee> salCompare = new Comparator<Employee>() {
            @Override
            public int compare(Employee e1, Employee e2) {
                    return Double.compare(e2.salary, e1.salary); // Descending
order by salary
            }
};
        // Creating an HashSet of Employee objects
        PriorityQueue<Employee> e = new PriorityQueue<Employee>(salCompare);
        // Adding Employee objects to the HashSet
        e.add(new Employee(1, "Saadvi", 40000));
        e.add(new Employee(2, "Suresh", 60000));
        e.add(new Employee(3, "Kalpana", 85000));
        System.out.println(e);
    }
}
```

SURESH YADLAPATI

Difference between ArrayList and LinkedList classes:

	ArrayList	**LinkedList**
Underlying Data Structure	Dynamic Array	Doubly Linked List
Access time	**O(1)** for random access via index	**O(n)** for random access (traversal needed)
Insertion (Middle)	**Shifting required**	**No shifting required**
Deletion (Middle)	**Shifting required**	**No shifting required**
Memory Usage	Requires less memory (only stores data)	Requires more memory (stores data + pointers)
Iteration	Faster due to contiguous memory access	Slower due to pointer traversal
Best Use Case	When frequent access or search operations are needed	When frequent insertions/deletions are needed
Resizing	Needs resizing when capacity is exceeded	No resizing needed

SURESH YADLAPATI